POLYMORPHISM IN PROGRAMMING: CONCEPTS AND PRACTICE

KEY DEFINITIONS

Polymorphism → Ability of a function/operator to behave differently based on context.

Compile-time Polymorphism → Polymorphism resolved during compilation (function/operator overloading).

KEY DEFINITIONS

- Runtime Polymorphism → Polymorphism resolved during execution using virtual functions.

- Function Overloading → Same function name with different parameters (type/number).

KEY DEFINITIONS (PART 2)

- Operator Overloading → Redefining operators to work with user-defined objects.

- Function Overriding → Redefining base class function in derived class with same signature.

KEY DEFINITIONS

Virtual Function → Function declared with virtual keyword to enable runtime polymorphism.

Early Binding → Function call resolved at compile time.

KEY DEFINITIONS

- Late Binding → Function call resolved at runtime using virtual functions.
- Base Class Pointer → Pointer of parent class used to access derived class objects.

DYNAMIC BINDING

Another name for runtime polymorphism. Dynamic binding refers to the process where the function call is resolved at runtime rather than compile time, allowing the correct method to be invoked based on the actual object type.

PRACTICE QUESTION 1

Question
Create a class Calculator that:

- Overloads add() for:
  - 2 integers
  - 3 integers
  - 2 doubles
- Overloads multiply() for:
  - 2 integers
  - 3 integers
- Also include:
  - subtract(int, int)
  - divide(double, double)

```
#include<iostream>
using namespace std;

class Calculator {
public:
 // Function Overloading - add
 int add(int a, int b) {
 return a + b;
 }

 int add(int a, int b, int c) {
 return a + b + c;
 }

 double add(double a, double b) {
 return a + b;
 }

 // Function Overloading - multiply
 int multiply(int a, int b) {
 return a * b;
 }

 int multiply(int a, int b, int c) {
 return a * b * c;
 }

 int subtract(int a, int b) {
 return a - b;
 }

 double divide(double a, double b) {
 if (b != 0)
 return a / b;
 else {
 cout << "Division by zero error\n";
 return 0;
 }
 }
};

int main() {
 Calculator c;

 cout << "Add(2,3): " << c.add(2,3) << endl;
 cout << "Add(2,3,4): " << c.add(2,3,4) << endl;
 cout << "Add(2.5,3.5): " << c.add(2.5,3.5) << endl;

 cout << "Multiply(2,3): " << c.multiply(2,3) << endl;
 cout << "Multiply(2,3,4): " << c.multiply(2,3,4) << endl;

 cout << "Subtract(10,5): " << c.subtract(10,5) << endl;
 cout << "Divide(10,2): " << c.divide(10,2) << endl;

 return 0;
}
```

PRACTICE QUESTION 2

Question
Create a class Complex:

- Overload:
  - + for addition
  - - for subtraction
- Display result using display()

```
#include<iostream>
using namespace std;
class Complex {
private:
 float real, imag;

public:
 Complex(float r=0, float i=0) {
 real = r;
 imag = i;
 }

 // Overload +
 Complex operator+(Complex c) {
   return Complex(real + c.real, imag + c.imag);
 }

 // Overload -
 Complex operator-(Complex c) {
 return Complex(real - c.real, imag - c.imag);
 }

void display() {
 cout << real << " + " << imag << "i" << endl;
 }
};


int main() {
 Complex c1(3,4), c2(1,2), result;

 result = c1 + c2;
 cout << "Addition: ";
 result.display();

 result = c1 - c2;
 cout << "Subtraction: ";
 result.display();

 return 0;
}
```

PRACTICE QUESTION 3

Question
Create base class Person:
- Virtual function Draw()

Derived classes:
- Artist
- Gunman

Use array of base class pointers.

```
#include<iostream>
using namespace std;

class Person {
public:
 virtual void Draw() {
 cout << "A person can draw in many ways\n";
 }
};

class Artist : public Person {
public:
 void Draw() {
  cout << "An artist can draw with a paint brush\n";
 }
};

class Gunman : public Person {
public:
 void Draw() {
 cout << "A gunman draws a gun to shoot\n";
 }
};
```

```
int main() {
 Person* arr[3];

 arr[0] = new Person();
 arr[1] = new Artist();
 arr[2] = new Gunman();

 for(int i=0; i<3; i++) {
 arr[i]->Draw(); // Runtime Polymorphism
 }

 return 0;
}
```

PRACTICE QUESTION 4

Question
Create class Number:

- Overload unary - to change sign

```
#include<iostream>
using namespace std;

class Number {
private:
 int x;

public:
 Number(int val) {
 x = val;
 }

 void operator-() {
 x = -x;
 }

 void display() {
 cout << x << endl;
 }
};
```

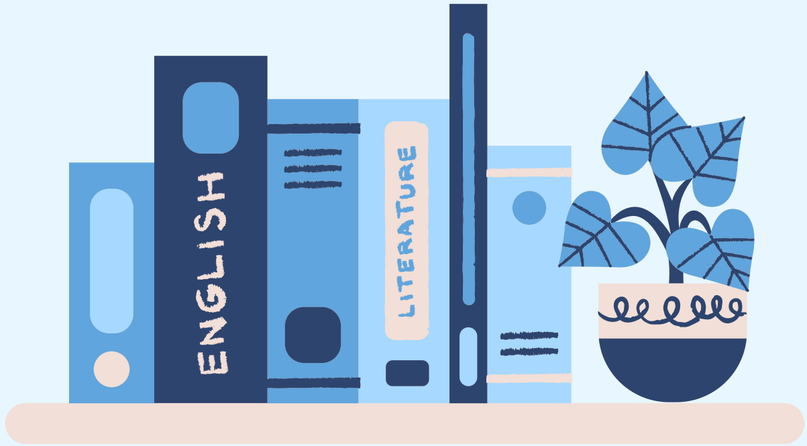
```
int main() {
 Number n(10);

 -n;
 n.display();

 return 0;
}
```

SCENERIO QUESTION

You are developing a 2D Game System with different characters and actions.

1. Create a base class Character:
   - Virtual functions attack() and power()
2. Create derived classes:
   - Warrior
   - Mage
   - (Override both functions)
3. Create class DamageCalculator:
   - Overload calculateDamage():
     - 1 parameter → basic attack
     - 2 parameters → special attack
4. Create class Position:
   - Data: x, y
   - Overload + to combine two positions
5. In main():
   - Use base class pointer to call attack() and power()
   - Demonstrate function overloading
   - Demonstrate operator overloading

Polymorphism in Programming

THANK YOU!

Any questions?